SÉCURITÉ DE L'INFRASTRUCTURE AS CODE

iac-guard

Documentation complète

Fonctionnement de bout en bout, protocoles, outils, algorithmes, méthode de travail et procédures opérationnelles.

DÉPÔT	**ZenithSoft / iac-guard** — branche **main**
ÉTAT DÉCRIT	commit **a2854f1** — « Refuser le démarrage en production avec une configuration dangereuse »
DATE	**15 septembre 2026**
VÉRIFICATION	Chiffres relus dans le code · suite de tests exécutée : **151 cas passants, 0 échec**

Sommaire

Carte des documents

1. Le produit en une page

2. Fonctionnement de bout en bout

- 2.1 Le schéma général
- 2.2 Le cycle de vie d'un scan, étape par étape
- 2.3 Le parcours d'un être humain

3. Les outils de la chaîne

- 3.1 Les scanners (exécutés dans le pipeline, chez le client)
- 3.2 Le socle backend
- 3.3 Le socle frontend
- 3.4 Infrastructure et outillage

4. Les protocoles

- 4.1 HTTP / REST — le transport de tout
- 4.2 Bearer token — l'authentification du pipeline
- 4.3 JWT (JSON Web Token) — la session humaine
- 4.4 Cookies — le stockage de la session
- 4.5 CORS — qui a le droit de parler à l'API
- 4.6 WebAuthn / FIDO2 — la biométrie et les clés de sécurité
- 4.7 TOTP — RFC 6238
- 4.8 Hachage et empreintes — quel algorithme, pour quoi
- 4.9 Le contrat pipeline → API : le schéma unifié v1.0
- 4.10 SMTP — l'exception qui n'en est pas une
- 4.11 Accès base de données

5. Architecture logicielle

- 5.1 Arborescence commentée
- 5.2 Les couches du backend et leurs responsabilités
- 5.3 Les trois circuits d'authentification

6. Modèle de données

- 6.1 Table par table

7. L'API, route par route

- 7.1 Méta
- 7.2 Authentification — /api/auth (10 routes)
- 7.3 Double authentification — /api/auth/2fa (12 routes)
- 7.4 Projets — /api/projects (9 routes)
- 7.5 Ingestion (1 route)
- 7.6 Lecture — scans, alertes, audit (6 routes)
- 7.7 Administration — /api/admin (9 routes)

8. Le tableau de bord

- 8.1 Les 9 routes de l'application
- 8.2 Les composants métier notables
- 8.3 Les modules lib/
- 8.4 Le panneau de détail d'un finding

9. Les algorithmes et règles métier

- 9.1 L'empreinte stable (fingerprint)
- 9.2 Le score de conformité
- 9.3 La vue « dernier état »
- 9.4 La politique de blocage
- 9.5 Le RBAC à cinq rôles
- 9.6 Les règles de protection des comptes
- 9.7 Le triage
- 9.8 Anti-force brute et anti-énumération
- 9.9 Les garde-fous de démarrage en production

10. Sécurité : menace par menace

- 10.1 Ce qu'il faut annoncer comme limites

11. La méthode de travail

- 11.1 Le principe directeur : le code explique ses décisions
- 11.2 Conventions de code
- 11.3 Les arbitrages d'ingénierie, et leur logique
- 11.4 La stratégie de test
- 11.5 La méthode de documentation
- 11.6 Le cycle de développement observable

12. Procédures opérationnelles

- 12.1 Installer l'environnement de développement
- 12.2 Développer sans Docker
- 12.3 Connecter un dépôt client
- 12.4 Faire tourner un jeton de projet
- 12.5 Inviter un collaborateur
- 12.6 Donner accès à un projet
- 12.7 Traiter une alerte
- 12.8 Déployer en production
- 12.9 Sauvegarder et restaurer
- 12.10 Diagnostiquer une panne

13. Tests

- 13.1 Résultat
- 13.2 Infrastructure
- 13.3 Répartition
- 13.4 Ce qui n'est pas couvert

14. Configuration de référence

- 14.1 Backend — classe Settings
- 14.2 Frontend
- 14.3 Ports et réseau

15. Limites connues et suite

- 15.1 Travaux en cours annoncés par le README
- 15.2 Autres éléments incomplets identifiés dans le code
- 15.3 Limites techniques structurelles
- 15.4 Ordre de priorité suggéré

16. Glossaire

Carte des documents

Ce dépôt contient cinq documents. Ils ne se remplacent pas, ils s'emboîtent :

DOCUMENT	RÉPOND À LA QUESTION	PUBLIC	TAILLE
`README.md`	« Comment je le lance ? »	Développeur qui découvre le dépôt	175 lignes
`PRESENTATION_PROJET.md`	« Pourquoi ce produit existe, que promet-il ? »	Jury, client, chef de projet	480 lignes
`PROJECT_ANALYSIS.md`	« Qu'y a-t-il exactement dans le code ? »	Relecteur technique, auditeur	1 476 lignes
`SYNTHESE_ANALYSE.md`	« Comment cette analyse a-t-elle été faite ? »	Encadrant, correcteur	114 lignes
`DOCUMENTATION_COMPLETE.md` *(ce fichier)*	**« Comment ça marche, avec quels protocoles, quels outils, et comment on travaille dessus ? »**	Toute personne qui doit reprendre, exploiter ou évaluer le projet	—

Documents annexes, dans `docs/` :

- `docs/speech-presentation.md` — texte de présentation orale (~1 min 30).
- `docs/script-video-demo.md` — script minuté d'une démonstration de 5 minutes.
- `docs/demo-vulnerable-main.tf` — code Terraform volontairement vulnérable (14 ressources) servant de cible de test.
- `docs/sample-scan.json` — exemple de rapport au schéma unifié.

1. Le produit en une page

iac-guard est un contrôle de sécurité automatique posé entre le développeur et la production.

Une équipe décrit son infrastructure en code (Terraform). Ce code part en production sans relecture sécurité. Trois familles de défauts passent alors :

1. **des secrets commités** — un mot de passe de base, une clé d'API — qui restent dans l'historique Git pour toujours ;
2. **des ressources mal configurées** — stockage public, base non chiffrée, port d'administration ouvert sur Internet ;
3. **des vulnérabilités applicatives** — injection SQL, injection de commande.

Des analyseurs existent pour chacune. Le problème n'est pas leur absence : c'est que **personne ne les lance**, qu'ils **produisent trois formats incompatibles**, et qu'ils **n'empêchent rien**.

iac-guard répond en cinq gestes :

GESTE	CE QUE ÇA VEUT DIRE CONCRÈTEMENT
Automatiser	Trois scanners se lancent seuls à chaque `push` et chaque *pull request*. Le développeur n'installe rien.
Unifier	Un agrégateur convertit trois formats JSON incompatibles en un schéma unique (`ScanReport`).
Centraliser	Tout arrive dans une API, est historisé, et s'affiche dans un tableau de bord multi-projets.
Faire décider	Chaque alerte se triage : *ouverte*, *prise en compte*, *résolue*, *faux positif* — avec justification et journal d'audit.
Bloquer	Au-delà du seuil de gravité choisi pour le projet, le *check* GitHub échoue et le bouton « Merge » est grisé.

Ce qui distingue le produit d'un simple lanceur de scanners : le blocage effectif de la fusion, le suivi d'un même problème dans le temps par empreinte stable, le triage tracé, et une chaîne d'authentification à trois circuits séparés.

2. Fonctionnement de bout en bout

2.1 Le schéma général



2.2 Le cycle de vie d'un scan, étape par étape

Étape 1 — Déclenchement

Le workflow `pipeline/github-actions-template/scan-standalone.yml` se déclenche sur :

- `push` limité aux chemins `**.tf`, `**.py`, `**.js`, `.github/workflows/scan.yml` ;

- toute `pull_request` (sans filtre de chemin) ;
- `workflow_dispatch` — le bouton « Run workflow » de l'onglet Actions.

Le `checkout` utilise `fetch-depth: 0`. Ce n'est pas un détail : sans l'historique complet, Gitleaks ne verrait que l'état actuel des fichiers et **manquerait un secret supprimé dans un commit ultérieur** — or c'est précisément le cas dangereux, puisque le secret reste récupérable dans l'historique.

Étape 2 — Contrôle de joignabilité

Avant de dépenser trois minutes de scan, le workflow appelle `GET {API_URL}/health`. L'URL est d'abord nettoyée de ses espaces et de son éventuel slash final. Un copier-coller depuis Windows colle facilement un `\r\n` invisible en fin de variable ; `curl` refuse alors l'URL avec un message (« URL rejected: Malformed input ») qui n'explique rien. Si l'API ne répond pas, le job s'arrête ici avec un message clair.

Étape 3 — Les trois scanners

Chaque étape porte `continue-on-error: true`. La raison est technique : **un scanner qui trouve des problèmes sort en code non nul**. Sans cette tolérance, le premier scanner qui trouve quelque chose arrêterait le job et on perdrait les deux autres rapports.

SCANNER	COMMANDE EXACTE	SORTIE
Gitleaks	`gitleaks detect --source . --report-format json --report-path gitleaks.json --redact --exit-code 0`	Liste plate de secrets
Checkov	`checkov -d . -o json --soft-fail > checkov.json`	`{results: {failed_checks: [...]}}`
Semgrep	`semgrep scan --config=auto --json --output semgrep.json`	`{results: [...]}`

`--redact` sur Gitleaks masque la valeur du secret dans le rapport : le secret ne transite donc **pas** vers l'API, seule sa localisation le fait.

Étape 4 — L'agrégation

`pipeline/aggregator/aggregator.py` lit les trois fichiers JSON, applique un *parser* par outil, et produit une liste de `Finding` homogènes. Un fichier absent ou vide donne simplement une liste vide — aucune erreur : un scanner peut avoir échoué sans faire tomber la chaîne.

Étape 5 — L'envoi

```
POST {API_URL}/api/scans
Authorization: Bearer iacg_...
Content-Type: application/json

{ "schema_version": "1.0", "scan": {...}, "summary": {...}, "findings": [ ... ] }
```

Cette étape, **elle, ne porte pas** `continue-on-error`. C'est délibéré et c'est écrit en commentaire dans le workflow : *« un pipeline vert alors que le scan s'est perdu en route est pire que pas de scan du tout »*. Si l'envoi échoue, le job échoue.

Étape 6 — Le verdict

L'API renvoie `should_fail: true|false`. L'agrégateur relaie ce verdict en code de sortie :

```
if result.get("should_fail"):
    print(f"[iac-guard] ÉCHEC DU PIPELINE : {result.get('fail_reason')}")
    return 1
```

Le job GitHub passe au rouge. Si la branche principale est protégée avec le *check* `scan` exigé, **le bouton « Merge » est grisé** : c'est le point où le produit cesse d'observer et commence à empêcher.

Étape 7 — Les rapports bruts

`actions/upload-artifact@v4` publie les trois JSON d'origine sur la page du run (`if: always()` , donc même en cas d'échec). On peut comprendre un finding sans relancer un scan complet.

2.3 Le parcours d'un être humain

```
Connexion
  POST /api/auth/methods   <- quelles methodes pour cet e-mail ?
        |
        +- passkey seule ---> POST /api/auth/webauthn/login/{options,verify}
        |                      puis cookie de SESSION directement
        |
        +- mot de passe ----> POST /api/auth/login
                              |
                              +- pas de 2FA ---> cookie de SESSION (24 h)
                              |
                              +- 2FA active ---> cookie MFA (5 min, portee « mfa »)
                                                  |
                                        +---------+---------+
                                        v         v         v
                                   webauthn/   totp/verify   recovery/verify
                                authenticate/verify
                                        +---------+---------+
                                                  |
                                            cookie de SESSION
```

Une fois la session ouverte, le tableau de bord consomme les routes de lecture, toutes cloisonnées par projet accessible.

3. Les outils de la chaîne

3.1 Les scanners (exécutés dans le pipeline, chez le client)

Gitleaks 8.21.2 — détection de secrets

- **Cherche** : clés d'API, jetons, mots de passe en dur, par expressions régulières et calcul d'entropie.
- **Portée** : l'arbre de travail **et tout l'historique Git** (grâce à `fetch-depth: 0`).
- **Installation** : téléchargement direct de la release GitHub, décompression dans `/usr/local/bin`. Aucun gestionnaire de paquets, aucune image Docker — la version est épinglée dans `env: GITLEAKS_VERSION`.
- **Format de sortie** : liste JSON plate. Champs exploités : `RuleID`, `Description`, `File`, `StartLine`, `EndLine`.
- **Choix produit** : Gitleaks ne déclare pas de sévérité. iac-guard tranche : **tout secret exposé est `critical`**. Un secret est soit dans le code soit il n'y est pas ; il n'y a pas de demi-mesure.
- **Remédiation injectée** : « Retirer le secret du code, le révoquer et le stocker dans un gestionnaire de secrets (Vault, SSM, GitHub Secrets). » — le mot important est *révoquer* : retirer le secret du code ne suffit pas, il reste dans l'historique.

Checkov — mauvaises configurations Infrastructure as Code

- **Cherche** : ressources cloud mal déclarées — bucket public, volume non chiffré, port d'administration ouvert, journalisation absente.
- **Installation** : `pip install checkov`, version **non épinglée** (voir §15, c'est une limite assumée).
- **Format de sortie** : `{results: {failed_checks: [...]}}`, ou une **liste** de tels objets quand plusieurs frameworks sont détectés. Le parser gère les deux cas.
- **Champs exploités** : `check_id` (ex. `CKV_AWS_21`), `check_name`, `severity`, `resource`, `repo_file_path`, `file_line_range`, `guideline`.
- **Choix produit** : Checkov ne renseigne `severity` que pour les abonnés à la plateforme Prisma. En open source, le champ est vide → `medium` **par défaut**.

Semgrep 1.78.0 — analyse statique du code applicatif (SAST)

- **Cherche** : injection SQL, injection de commande, désérialisation dangereuse, etc., dans le code applicatif (Python, JavaScript…), pas dans l'IaC.
- **Configuration** : `--config=auto`, c'est-à-dire le jeu de règles du Semgrep Registry adapté aux langages détectés dans le dépôt.
- **Format de sortie** : `{results: [...]}`. Champs exploités : `check_id`, `extra.message` (première ligne seulement, comme titre), `extra.severity`, `extra.fix`, `path`, `start.line`, `end.line`.
- **Choix produit** : Semgrep exprime la gravité en `ERROR` / `WARNING` / `INFO`, pas en critical/high/medium/low. La table de correspondance mappe `error → high` et `warning → medium`.

Conftest 0.56.0 / OPA — règles métier (support présent, inactif)

- **Statut** : le parser existe et est complet dans l'agrégateur, l'énumération `ToolName.CONFTEST` existe, le workflow `scan.yml` l'appelle. **Mais le dossier `policies/` du dépôt est vide** : il n'y a aucune règle Rego à

charger, et le workflow réellement recommandé par le README (`scan-standalone.yml`) n'appelle pas Conftest. Le support est donc du code prêt, pas une fonctionnalité livrée.

- **Ce que le parser sait faire** : lire `failures` (défaut `high`) et `warnings` (défaut `low`), et laisser la métadonnée Rego surcharger `rule_id`, `title`, `severity`, `resource`, `remediation`. Le choix de conception est que **la règle Rego porte sa propre sévérité**.

3.2 Le socle backend

OUTIL	VERSION	RÔLE
Python	3.12	Langage du backend et de l'agrégateur
FastAPI	≥0.115, <1	Framework HTTP, validation, OpenAPI généré (`/docs`)
Uvicorn	≥0.30, <1	Serveur ASGI (`--host 0.0.0.0 --port 8000`)
SQLAlchemy	≥2.0, <3	ORM, style 2.0 (`Mapped[...]` , `select()`)
Pydantic	≥2.10, <3	Schémas d'entrée/sortie, validation
pydantic-settings	≥2.5, <3	Configuration par variables d'environnement + `.env`
python-jose[cryptography]	≥3.3, <4	Signature et vérification des JWT (HS256)
argon2-cffi	≥23.1, <26	Hachage des mots de passe (Argon2id)
bcrypt	≥4.1, <6	**Lecture seule** : vérifier les anciens hash pour les migrer
webauthn	≥2.0, <3	Génération/vérification des challenges FIDO2
pyotp	≥2.9, <3	TOTP (RFC 6238)
email-validator	≥2.0, <3	Validation des `EmailStr` Pydantic
psycopg[binary]	≥3.2, <4	Pilote PostgreSQL
pytest	≥8.0, <9	Tests (dev uniquement)
httpx	≥0.27, <1	Requis par `fastapi.testclient`

Note sur `passlib` : volontairement **absent**. Le commentaire de `requirements.txt` explique pourquoi — passlib n'est plus maintenu et lit `bcrypt.__about__` , supprimé depuis bcrypt 4.1 ; il casse à l'exécution. bcrypt est donc appelé directement.

3.3 Le socle frontend

OUTIL	VERSION	RÔLE
Next.js	16.2.10	Framework React, App Router, Turbopack
React / React DOM	19.2.4	Bibliothèque d'interface

OUTIL	VERSION	RÔLE
TypeScript	^5	Typage
Tailwind CSS	^4 (via `@tailwindcss/postcss`)	Styles utilitaires
shadcn	^4.13.0	Générateur de composants (22 composants dans `components/ui/`)
@base-ui/react	^1.6.0	Primitives accessibles sous-jacentes
@simplewebauthn/browser	^13.3.0	Appels `navigator.credentials.*` côté navigateur
@tanstack/react-table	^8.21.3	Table d'alertes : tri, pagination
recharts	^3.8.0	Courbe d'activité
qrcode.react	^4.2.0	QR code d'enrôlement TOTP
input-otp	^1.4.2	Champ de saisie du code à 6 chiffres
sonner	^2.0.7	Notifications *toast*
next-themes	^0.4.6	Thème clair / sombre
lucide-react	^1.24.0	Icônes
clsx / tailwind-merge / class-variance-authority	—	Composition de classes CSS
eslint / eslint-config-next	^9 / 16.2.10	Analyse statique du front

3.4 Infrastructure et outillage

OUTIL	DÉTAIL
Docker / Docker Compose	Deux services : `db` (postgres:16-alpine, volume `pgdata`, *healthcheck* `pg_isready`) et `api` (build de `./backend`, démarre seulement quand la base est saine)
PostgreSQL	16-alpine en développement conteneurisé ; SQLite en développement local
GitHub Actions	Runner `ubuntu-latest`. Actions utilisées : `actions/checkout@v4`, `actions/setup-python@v5`, `actions/upload-artifact@v4`
Git	4 commits sur `main`, messages en français, à l'impératif
pytest	`pytest.ini` : `pythonpath = .`, `testpaths = tests`, `addopts = -q`
MCP shadcn	`.mcp.json` déclare un serveur MCP `shadcn` (`npx shadcn@latest mcp`) utilisé pendant le développement de l'interface

4. Les protocoles

Cette section liste les protocoles et standards effectivement mis en œuvre, avec **où** et **comment**.

4.1 HTTP / REST — le transport de tout

- **Style** : REST sur JSON. Les ressources sont `/api/projects`, `/api/scans`, `/api/alerts`, `/api/admin/users`…
- **Verbes** : `GET` (lecture), `POST` (création et actions), `PATCH` (modification partielle), `PUT` (remplacement complet — utilisé pour l'attribution de projets, qui est un *set*), `DELETE`.
- **Codes de statut employés** : `200`, `201` (création), `202` (accepté sans effet garanti — mot de passe oublié), `204` (succès sans corps), `401` (non authentifié), `403` (rôle insuffisant / compte désactivé), `404` (introuvable **ou** hors périmètre), `409` (conflit métier), `422` (validation), `429` (verrouillage / quota).
- **Documentation** : OpenAPI générée automatiquement par FastAPI, servie sur `/docs` (Swagger UI) et `/openapi.json`.
- **Sonde de vie** : `GET /health` → `{"status": "ok", "app": "iac-guard"}`. C'est ce que le pipeline appelle avant de scanner et ce que Compose peut interroger.

4.2 Bearer token — l'authentification du pipeline

En-tête standard `Authorization: Bearer <token>` (RFC 6750). Le jeton a la forme `iacg_` + 32 octets aléatoires encodés en URL-safe base64 (`secrets.token_urlsafe(32)`).

Côté serveur, le jeton **n'est jamais stocké en clair** :

```
project = db.scalar(select(Project).where(Project.token_hash == hash_token(token)))
```

`hash_token` est un SHA-256 simple, **sans sel et sans dérivation lente** — et c'est justifié : le jeton porte 256 bits d'entropie aléatoire, il n'est donc pas attaquable par dictionnaire. Ajouter Argon2 ici ne protégerait rien et coûterait une vérification lente à chaque scan.

Ce qui est stocké en plus : `token_prefix`, les neuf premiers caractères (`iacg_ab12…`), uniquement pour que l'interface puisse afficher de quel jeton on parle.

Conséquence assumée : un jeton perdu n'est pas récupérable. On en régénère un (§12.4).

4.3 JWT (JSON Web Token) — la session humaine

- **Algorithme** : HS256 (HMAC-SHA256), clé symétrique `SECRET_KEY`.
- **Charge utile** :

`json { "sub": "42", "scope": "session", "iat": 1757... , "exp": 1757... }`

`sub` est une **chaîne** (la spécification JWT l'impose), même si l'identifiant utilisateur est un entier.

- **Deux portées, et c'est le point central** :

PORTÉE	DURÉE	CE QU'ELLE OUVRE
`session`	`access_token_expire_minutes` = 1 440 min (24 h)	L'API complète
`mfa`	`mfa_token_expire_minutes` = 5 min	**Uniquement** les routes de vérification du second facteur

`decode_access_token(token, expected_scope)` refuse un jeton dont la portée ne correspond pas. Sans ce contrôle, le jeton délivré après le mot de passe mais avant la 2FA ouvrirait l'API — **la 2FA deviendrait contournable**. C'est la ligne la plus importante du module.

- **Ce que le JWT ne contient pas** : le rôle. Il est relu en base à chaque requête (§9.5). Un changement de rôle prend donc effet immédiatement, sans attendre l'expiration ni gérer une liste de révocation.

4.4 Cookies — le stockage de la session

Deux cookies, posés par `app/api/cookies.py` :

COOKIE	CONTENU	DURÉE
`iacguard_session`	JWT de portée `session`	24 h
`iacguard_mfa`	JWT de portée `mfa`	5 min

Attributs, et la raison de chacun :

ATTRIBUT	VALEUR	POURQUOI
`httponly`	`True`	JavaScript ne peut pas lire le cookie. Une faille XSS sur le tableau de bord **ne peut donc pas voler la session**. En `localStorage`, le moindre script injecté la vole.
`secure`	`settings.cookie_secure` — `False` en dev, **obligatoirement** `True` **en production**	Impose HTTPS. Le démarrage est refusé en production si ce n'est pas le cas.
`samesite`	`lax`	Le navigateur n'envoie pas le cookie sur une requête POST venue d'un autre site : l'essentiel des attaques CSRF tombe.
`path`	`/`	Portée à toute l'API.
`max_age`	Aligné sur l'expiration du JWT	Le cookie et le jeton meurent ensemble.

Poser le cookie de session efface automatiquement le cookie MFA : il a rempli son rôle.

Les clients non-navigateur peuvent utiliser `Authorization: Bearer <JWT>` à la place du cookie ; `get_current_user` accepte les deux.

4.5 CORS — qui a le droit de parler à l'API

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins,   # liste EXPLICITE
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

`allow_credentials=True` est indispensable pour que le navigateur joigne le cookie. Il **interdit** `allow_origins=["*"]` — et c'est précisément la garantie recherchée : un site tiers ne peut pas appeler l'API avec le cookie de la victime. La liste par défaut est `["http://localhost:5173", "http://localhost:3000"]`.

4.6 WebAuthn / FIDO2 — la biométrie et les clés de sécurité

Le standard W3C WebAuthn, avec CTAP2 entre le navigateur et l'authentificateur. Implémenté dans `app/services/mfa.py` (py_webauthn) et côté navigateur via `@simplewebauthn/browser`.

Le principe en une phrase : le serveur envoie un défi aléatoire, l'appareil le signe avec une clé privée qui ne quitte jamais le TPM ou la Secure Enclave (déverrouillée par l'empreinte, le visage ou le PIN), et le serveur vérifie la signature avec la clé publique. **Le serveur ne voit jamais la biométrie.**

Deux cérémonies :

```
ENROLEMENT (register)                 CONNEXION (authenticate)
--------------------                  ------------------------
POST .../register/options             POST .../authenticate/options
  serveur -> options + challenge        serveur -> options + challenge
  (stocke le challenge en base)         (stocke le challenge en base)
        |                                     |
navigator.credentials.create()        navigator.credentials.get()
  l'appareil genere une paire           l'appareil signe le challenge
  de cles, garde la privee              avec sa cle privee
        |                                     |
POST .../register/verify              POST .../authenticate/verify
  serveur verifie et stocke             serveur verifie la signature
  la CLE PUBLIQUE                       avec la cle publique stockee
```

Paramètres retenus et leur raison :

PARAMÈTRE	VALEUR	RAISON
`rp_id`	`webauthn_rp_id`, par défaut `localhost`	Domaine « propriétaire » de la clé. Ce doit être le domaine du **frontend**, pas celui de l'API. C'est ce qui rend les passkeys résistantes au *phishing* : une clé créée pour un domaine ne peut pas être présentée à un autre.
`expected_origin`	`webauthn_origins`, liste exacte	Origine autorisée à présenter une assertion.
`user_verification`	`REQUIRED`	L'appareil **doit** vérifier l'utilisateur (empreinte, visage ou PIN). Sans cela une simple présence suffirait : ce ne serait pas un vrai second facteur.
`resident_key`	`PREFERRED`	Permet la connexion sans saisir d'e-mail (*passkey découvrable*), sans l'imposer aux authentificateurs qui ne la gèrent pas.
`exclude_credentials`	Clés déjà enrôlées	Empêche d'enrôler deux fois le même appareil.

Challenge : stocké **en base** (table `auth_challenges`), pas dans un cookie, et **supprimé à la première utilisation** (`_consume_challenge` lit puis `db.delete`). Un challenge doit être à usage unique, sinon une signature interceptée pourrait être rejouée. TTL : 300 secondes. Un seul challenge en cours par couple *(utilisateur, usage)* — le nouveau écrase l'ancien.

Compteur anti-clonage : l'authentificateur incrémente `sign_count` à chaque usage. S'il régresse, la clé a probablement été clonée → refus. Nuance implémentée : beaucoup de passkeys renvoient toujours 0 ; dans ce cas on ne peut rien déduire et le contrôle n'est pas appliqué.

```
if verified.new_sign_count != 0 and verified.new_sign_count <= stored.sign_count:
    raise MfaError("Clé potentiellement clonée : authentification refusée.")
```

Gestion d'erreurs : `WebAuthnException`, la classe mère, est capturée en entier plutôt que chaque sous-classe — parsing JSON, signature invalide, CBOR corrompu… en oublier une donnerait un HTTP 500 au lieu d'un message clair.

4.7 TOTP — RFC 6238

Le code à six chiffres des applications d'authentification (Google Authenticator, Authy, Microsoft Authenticator).

- **Secret** : base32 aléatoire, `pyotp.random_base32()`.
- **Transmission au téléphone** : URI `otpauth://` encodée en QR code côté navigateur (`qrcode.react`), construite par `provisioning_uri(name=user.email, issuer_name="iac-guard")`.
- **Fenêtre de tolérance** : `valid_window=1`, soit ±30 secondes. Les horloges des téléphones dérivent ; sans cette tolérance, des codes valides seraient refusés.
- **Activation en deux temps** : `POST /totp/setup` génère et stocke le secret, mais `totp_enabled` reste `False`. Il ne passe à `True` qu'après `POST /totp/confirm` avec un premier code valide — preuve que l'application est bien configurée. **Sans ce garde-fou, on enfermerait l'utilisateur dehors.**
- **Différence de nature avec WebAuthn** : le secret TOTP est *partagé* entre le serveur et le téléphone. Une fuite de base compromet donc les secrets TOTP, pas les clés WebAuthn (dont seule la partie publique est stockée).

C'est écrit noir sur blanc dans le docstring du modèle.

4.8 Hachage et empreintes — quel algorithme, pour quoi

Le projet emploie **quatre** primitives différentes, chacune choisie pour une raison précise. C'est un des points les plus soignés du code.

CE QU'ON PROTÈGE	ALGORITHME	POURQUOI CELUI-LÀ
Mot de passe utilisateur	**Argon2id** (paramètres par défaut d'argon2-cffi, alignés OWASP : m=64 MiB, t=3, p=4 ; sel embarqué)	Choisi par un humain, donc devinable. Argon2id est *memory-hard* : il impose beaucoup de RAM au vérificateur, ce qui rend le cassage massif sur GPU/ASIC bien plus coûteux que bcrypt.
Anciens mots de passe	**bcrypt**, en lecture seule	Migration transparente : au premier login réussi, `needs_rehash` détecte le préfixe `$2` et re-hache en Argon2id. Le mot de passe en clair n'existe qu'à cet instant précis.
Jeton de projet, jeton d'invitation, code de récupération	**SHA-256** sans sel	Ces valeurs sont générées aléatoirement (≥128 bits), donc non attaquables par dictionnaire. Une dérivation lente n'apporterait rien.
Empreinte d'un finding	**SHA-1**	Ce n'est pas une primitive de sécurité mais un **identifiant de déduplication**. La résistance aux collisions adverses n'est pas requise ici : personne ne gagne rien à faire collisionner deux findings.

Détails défensifs autour des mots de passe :

- `MAX_PASSWORD_BYTES = 1024` — au-delà, refus. Sans borne, hacher 1 Mo de « a » transformerait la page de connexion en déni de service.
- `BCRYPT_MAX_BYTES = 72` — bcrypt ignore silencieusement les octets au-delà de 72. Le code **refuse** plutôt que de tronquer : sinon deux mots de passe différents ouvriraient le même compte.
- `verify_password` ne lève jamais : signature invalide, hash corrompu, format inconnu → `False`.
- Les codes de récupération sont comparés avec `hmac.compare_digest`, en temps constant, pour ne pas fuiter d'information par mesure de durée.

4.9 Le contrat pipeline → API : le schéma unifié v1.0

Défini dans `pipeline/aggregator/schema.py`. **C'est la source de vérité du contrat.** Le module est volontairement autonome — aucune dépendance au backend — pour pouvoir être copié dans le dépôt d'un client sans y tirer tout le SaaS.

```
ScanReport
├── schema_version : "1.0"
├── scan : ScanMeta
│   ├── repo        : "owner/repo"
│   ├── branch      : str | null
│   ├── commit_sha  : str | null
│   ├── created_at  : datetime (UTC)
│   └── tools       : [ ToolInfo { name, version } ]
├── summary : Summary
│   ├── total       : int
│   └── by_severity : { critical: n, high: n, medium: n, low: n, info: n }
└── findings : [ Finding ]
    ├── source       : gitleaks | checkov | conftest | semgrep
    ├── category     : secret | misconfiguration | policy | vulnerability
    ├── rule_id      : "CKV_AWS_21", id Gitleaks, nom de policy Rego...
    ├── title        : str
    ├── severity     : critical | high | medium | low | info
    ├── resource     : "aws_s3_bucket.data" | null
    ├── file         : str | null
    ├── line_start   : int | null
    ├── line_end     : int | null
    ├── remediation  : str | null
    ├── raw          : objet brut de l'outil (audit / debug)
    └── fingerprint  : calcule, voir §9.1
```

Trois décisions structurantes :

1. `category` **est déterminée par l'outil, pas devinée.** Gitleaks ⇒ `secret`, Checkov ⇒ `misconfiguration`, Conftest ⇒ `policy`, Semgrep ⇒ `vulnerability`.
2. `raw` **conserve l'objet d'origine**, intégralement, pour l'audit et le débogage. Rien n'est perdu à la normalisation.
3. **Le mapping réglementaire (CNDP, Bank Al-Maghrib) n'est pas dans ce schéma.** Le pipeline n'envoie que des faits techniques ; l'enrichissement réglementaire est prévu côté backend, à partir de `rule_id`. Cela évite de redéployer les pipelines clients à chaque évolution du référentiel.

4.10 SMTP — l'exception qui n'en est pas une

`app/services/email.py` **ne parle à aucun serveur SMTP**. Le lien d'invitation est écrit dans les journaux de l'API. C'est explicite et assumé le temps qu'un service transactionnel soit configuré ; c'est listé dans les travaux en cours du README et au §15 de ce document.

4.11 Accès base de données

- **URL** : `sqlite:///./iacguard.db` par défaut ; `postgresql+psycopg://user:pass@host:5432/iacguard` en production.

- **SQLite** : `check_same_thread=False` est nécessaire pour être utilisé depuis FastAPI (multi-thread). Le flag n'est posé que si l'URL commence par `sqlite`.
- **Session** : une session SQLAlchemy par requête, injectée par la dépendance `get_db`, fermée dans un `finally`.
- **Schéma** : `Base.metadata.create_all(bind=engine)` au démarrage (`lifespan`). C'est un choix de MVP explicitement temporaire — Alembic est prévu (§15).

5. Architecture logicielle

5.1 Arborescence commentée

```
iac-guard/
├── backend/                    API FastAPI
│   ├── app/
│   │   ├── main.py             point d'entree : CORS, lifespan, 7 routeurs
│   │   ├── core/
│   │   │   ├── config.py       Settings + garde-fous de production
│   │   │   └── security.py     mots de passe, JWT, jetons, codes de secours
│   │   ├── db/session.py       engine, SessionLocal, Base, get_db
│   │   ├── models/             12 fichiers -> 14 tables SQLAlchemy
│   │   ├── schemas/            5 fichiers  -> schemas Pydantic d'E/S
│   │   ├── api/                6 modules de routes + deps.py + cookies.py
│   │   └── services/           scoring, aggregates, mfa, email, admin_audit
│   ├── tests/                  11 fichiers, 131 fonctions, 151 cas
│   ├── scripts/migrate_rbac.py migration ponctuelle vers le RBAC
│   ├── Dockerfile              python:3.12-slim + uvicorn
│   ├── requirements.txt        dependances de production
│   ├── requirements-dev.txt    + pytest, httpx
│   └── .env.example            modele de configuration commente
├── frontend/                   tableau de bord Next.js
│   └── src/
│       ├── app/                9 routes App Router
│       ├── components/         18 composants metier + 22 primitives ui/
│       ├── lib/                clients d'API, i18n, explications, utilitaires
│       └── hooks/use-mobile.ts
├── pipeline/
│   ├── aggregator/             schema.py (contrat) + aggregator.py + samples/
│   └── github-actions-template/ scan-standalone.yml (recommande) + scan.yml
├── policies/                   (vide — regles Rego a ecrire)
├── docs/                       demo Terraform, script video, speech, exemple JSON
├── docker-compose.yml          postgres:16-alpine + api
└── *.md                        les cinq documents (voir « Carte des documents »)
```

5.2 Les couches du backend et leurs responsabilités

COUCHE	DOSSIER	RESPONSABILITÉ	CE QU'ELLE NE FAIT PAS
Routes	`app/api/`	Recevoir, valider via Pydantic, orchestrer, répondre	Aucune logique de calcul métier

COUCHE	DOSSIER	RESPONSABILITÉ	CE QU'ELLE NE FAIT PAS
Dépendances	`app/api/deps.py`	Authentifier, autoriser, cloisonner	Rien d'autre : c'est le point de passage obligé
Services	`app/services/`	Calcul pur et logique réutilisable (score, agrégats, MFA, audit)	Ne connaissent pas HTTP
Modèles	`app/models/`	Structure des tables et relations	Pas de logique métier, sauf propriétés dérivées
Schémas	`app/schemas/`	Contrat d'entrée/sortie de l'API	Distincts des modèles : ce qu'on expose n'est pas ce qu'on stocke
Core	`app/core/`	Configuration et primitives de sécurité	Indépendant du framework web

Cette séparation a une conséquence concrète : `compute_score` est une fonction pure de `dict[str, int]` vers `ScoreResult`. Elle se teste sans base de données, sans client HTTP, sans *fixture* — d'où sept tests unitaires directs.

5.3 Les trois circuits d'authentification

C'est la décision d'architecture la plus structurante, documentée en tête de `deps.py`.

CIRCUIT	QUI	SUPPORT	DÉPENDANCE	PORTÉE
Token de projet	Le pipeline GitHub Actions	En-tête `Authorization: Bearer iacg_…`	`get_project_from_token`	Une seule route : `POST /api/scans`
JWT de session	Un humain connecté	Cookie `iacguard_session` (ou en-tête Bearer)	`get_current_user`	Toute l'API applicative
JWT MFA	Un humain à mi-connexion	Cookie `iacguard_mfa`	`get_mfa_user`	**Uniquement** les routes de vérification 2FA

Trois circuits séparés, trois surfaces d'attaque séparées. Un jeton de projet volé ne donne accès à aucune donnée du tableau de bord : il ne sait qu'écrire un scan dans un projet. Un jeton MFA volé ne donne accès à rien du tout tant que le second facteur n'est pas fourni.

6. Modèle de données

14 tables. Le graphe :

```
organizations
   |
   +---< users >---+---< projects >---< scans >---< findings
   |     |         |        |
   |     |         |        +---< finding_triage      (par empreinte)
   |     |         |        +---< ignored_rules       (exceptions de politique)
   |     |         |        +---< triage_audit_log    (append-only)
   |     |         |
   |     |         +---< project_access               (partage explicite)
   |     |
   |     +---< webauthn_credentials
   |     +---< recovery_codes
   |     +---< auth_challenges
   |
   +---< invitations
   +---< admin_audit_logs                             (append-only)
```

6.1 Table par table

`organizations` — le compte client

`id`, `name`, `created_at`. Périmètre d'administration : un admin ne gère que les membres de **son** organisation. L'organisation ne possède pas les projets — ceux-ci gardent un propriétaire individuel.

`users` — un humain du tableau de bord

Champs notables :

CHAMP	DÉTAIL
`email`	Unique, indexé
`hashed_password`	`NULL` tant qu'une invitation est en attente. **Le compte existe dès l'envoi de l'invitation.**
`role`	Un des cinq rôles, `admin` par défaut
`org_id`	Rattachement
`created_by_id`	`NULL` pour le premier compte d'une organisation (auto-inscription), sinon l'admin qui a invité
`totp_secret` / `totp_enabled`	Le secret est stocké dès le *setup*, mais l'activation attend un premier code valide

CHAMP	DÉTAIL
`failed_login_attempts` / `locked_until`	Anti-force brute
`last_login_at`	Alimenté à chaque connexion réussie

Quatre propriétés **dérivées** plutôt que stockées, pour éviter toute désynchronisation :

```
status                    -> "pending_invitation" | "active" | "disabled"
has_global_project_access -> role in ("admin", "security_lead")
webauthn_enabled          -> len(credentials) > 0
has_mfa / mfa_methods     -> etat reel de la 2FA
```

`projects` — un dépôt suivi

`name`, `owner_id`, `token_hash` (unique), `token_prefix`, `fail_on_severity` (nullable), `created_at`. Contrainte : `UniqueConstraint(owner_id, name)` — deux projets d'un même utilisateur ne peuvent pas porter le même nom. Cascades : supprimer un projet supprime ses scans, leurs findings et ses exceptions de règles.

`scans` — un rapport reçu

`project_id`, `schema_version`, `repo`, `branch`, `commit_sha`, `scanned_at` (instant côté pipeline), `received_at` (instant côté API), `total_findings`. **Deux horodatages distincts** : ils divergent quand un runner est lent ou que l'API a été indisponible.

`findings` — un problème détecté

`scan_id`, `fingerprint` (indexé), `source`, `category`, `rule_id`, `title`, `severity` (indexé), `resource`, `file`, `line_start`, `line_end`, `remediation`, `raw` (JSON).

`finding_triage` — l'état actuel d'une décision

`project_id`, `fingerprint`, `status`, `note`, `updated_by_id`, `updated_at`. `UniqueConstraint(project_id, fingerprint)` : une seule décision par problème et par projet. **Attaché à l'empreinte, pas à une ligne de scan** — c'est ce qui fait survivre la décision à la redétection.

`triage_audit_log` — l'historique des décisions (append-only)

`project_id`, **`project_name`**, **`owner_id`**, `finding_fingerprint`, **`finding_title`**, `from_status`, `to_status`, `note`, `user_id`, `created_at`.

Les champs en gras sont **dénormalisés volontairement**. La raison est écrite dans le modèle : *« un journal d'audit ne doit jamais dépendre de la survie de ce qu'il audite »*. Le projet peut être supprimé, l'entrée d'audit reste lisible et filtrable.

`admin_audit_logs` — l'historique des actions d'administration (append-only)

`org_id`, `actor_id`, `action`, `target_user_id`, `details` (JSON libre), `created_at`. Actions tracées : `user.invited`, `user.role_changed`, `user.enabled`, `user.disabled`, `user.webauthn_reset`, changement d'accès aux projets.

Deux journaux distincts, deux publics : ici *« qui a changé quoi sur quel compte »*, là *« qui a résolu quelle alerte »*.

`ignored_rules` — une exception de politique

`project_id`, `rule_id`, `note` (justification), `created_by_id`, `created_at`. `UniqueConstraint(project_id, rule_id)`. Différent du triage : ici on exclut **toute occurrence** d'une règle sur un projet, quelle que soit la ressource. C'est une exception de politique, pas un jugement au cas par cas.

`project_access` — un partage explicite

`user_id`, `project_id`, `granted_by_id`, `granted_at`. `UniqueConstraint(user_id, project_id)`. Ne concerne que les rôles sans accès global. **Règle produit** : un utilisateur à accès non global sans aucune ligne ici ne voit **aucun** projet.

`invitations` — la preuve d'une invitation

`email`, `org_id`, `role`, `user_id`, `token_hash` (unique), `invited_by_id`, `expires_at`, `accepted_at`, `created_at`. Le jeton en clair n'est **jamais** stocké ni renvoyé par l'API : il n'existe que dans le lien.

`webauthn_credentials` — une passkey

`user_id`, `credential_id` (binaire, unique), `public_key` (binaire), `sign_count`, `name` (« PC du bureau »), `transports`, `created_at`, `last_used_at`. Un utilisateur peut enrôler plusieurs appareils.

`recovery_codes` — un code de secours

`user_id`, `code_hash`, `used_at`. `used_at` non nul ⇒ code consommé. Sans ces codes, perdre son téléphone reviendrait à perdre son compte.

`auth_challenges` — un défi WebAuthn en attente

`user_id`, `challenge` (binaire), `purpose` (`register` | `authenticate`), `expires_at`. Supprimé dès consommation.

7. L'API, route par route

48 routes : 47 sous `/api`, plus `/health`.

7.1 Méta

MÉTHODE	ROUTE	AUTH	RÉPONSE
`GET`	`/health`	aucune	`{"status": "ok", "app": "iac-guard"}`
`GET`	`/docs`	aucune	Swagger UI (généré par FastAPI)

7.2 Authentification — `/api/auth` (10 routes)

MÉTHODE	ROUTE	AUTH	RÔLE
`POST`	`/register`	—	Crée un compte **et une organisation** dont il devient l'admin
`POST`	`/login`	—	Mot de passe → cookie de session, ou cookie MFA si 2FA active
`POST`	`/methods`	—	Étape 1 : quelles méthodes proposer pour cet e-mail (réponse identique si inconnu)
`POST`	`/webauthn/login/options`	—	Défi de connexion par passkey, **premier facteur**
`POST`	`/webauthn/login/verify`	—	Vérifie l'assertion et ouvre directement la session
`POST`	`/forgot-password`	—	Toujours `202`, aucun effet aujourd'hui (§15)
`POST`	`/logout`	—	Efface le cookie (`204`)
`GET`	`/me`	session	Profil de l'utilisateur connecté
`PATCH`	`/me`	session	Change nom et/ou e-mail
`POST`	`/me/password`	session	Change le mot de passe — **exige l'actuel**

Pourquoi `/me/password` exige le mot de passe actuel : sinon une session volée suffirait à verrouiller définitivement le compte de la victime.

7.3 Double authentification — `/api/auth/2fa` (12 routes)

MÉTHODE	ROUTE	AUTH	RÔLE
`GET`	`/status`	session	Méthodes actives, appareils enrôlés, codes restants

MÉTHODE	ROUTE	AUTH	RÔLE
`POST`	`/webauthn/register/options`	session	Options pour `navigator.credentials.create()`
`POST`	`/webauthn/register/verify`	session	Enregistre la clé **publique**
`DELETE`	`/webauthn/{credential_id}`	session	Retire un appareil
`POST`	`/webauthn/authenticate/options`	**mfa**	Défi à signer
`POST`	`/webauthn/authenticate/verify`	**mfa**	Vérifie et ouvre la session
`POST`	`/totp/setup`	session	Génère le secret + l'URI `otpauth://` (pas encore actif)
`POST`	`/totp/confirm`	session	Active après un premier code valide
`DELETE`	`/totp`	session	Désactive le TOTP
`POST`	`/totp/verify`	**mfa**	Second facteur par code à 6 chiffres
`POST`	`/recovery/verify`	**mfa**	Second facteur par code de secours (usage unique)
`POST`	`/recovery/regenerate`	session	Régénère les 8 codes, invalide les anciens

Noter la répartition des portées : l'enrôlement demande une **session** complète, la vérification à la connexion n'accepte qu'un jeton **mfa**.

7.4 Projets — `/api/projects` (9 routes)

MÉTHODE	ROUTE	RÔLE
`POST`	``	Crée un projet, **renvoie le jeton en clair une seule fois**
`GET`	``	Liste les projets accessibles (sans jetons)
`GET`	`/{id}`	Nombre de scans, score du dernier scan, date du dernier scan
`DELETE`	`/{id}`	Supprime le projet et tout son historique
`POST`	`/{id}/rotate-token`	Invalide l'ancien jeton, en émet un nouveau (affiché une fois)
`GET`	`/{id}/policy`	Seuil de blocage + exceptions de règles
`PATCH`	`/{id}/policy`	Change le seuil (`null` = ne jamais bloquer)
`POST`	`/{id}/ignored-rules`	Ajoute une exception, avec justification
`DELETE`	`/{id}/ignored-rules/{rid}`	Réactive la règle : ses findings recomptent

7.5 Ingestion (1 route)

MÉTHODE	ROUTE	AUTH	RÔLE
`POST`	`/api/scans`	**Token de projet**	Reçoit un `ScanReport`, persiste, renvoie le verdict

Le projet est identifié **par le jeton**, jamais par le contenu du message. Un client ne peut donc écrire que dans son propre projet, même s'il ment sur `repo`.

Réponse :

```
{
  "scan_id": 128,
  "project_id": 10,
  "total_findings": 23,
  "should_fail": true,
  "fail_reason": "Politique du projet : blocage dès qu'un finding atteint la gravité 'high' ou pire."
}
```

7.6 Lecture — scans, alertes, audit (6 routes)

MÉTHODE	ROUTE	RÔLE
`GET`	`/api/audit-log`	Journal de triage, paginé et filtrable
`GET`	`/api/projects/{id}/scans`	Historique des scans, du plus récent au plus ancien
`GET`	`/api/alerts`	**Vue « dernier état »**, tous projets confondus (voir §9.3)
`GET`	`/api/scans/{id}`	Un scan avec son score calculé
`GET`	`/api/scans/{id}/findings`	Les findings d'un scan, statut de triage inclus
`PATCH`	`/api/scans/{id}/findings/{fid}/triage`	Enregistre une décision

Filtres de `/api/alerts` : `project_id`, `severity`, `category`, `status`, `q` (recherche libre sur titre, ressource, fichier, `rule_id`), `page`, `page_size` (1 à 100, défaut 20).

7.7 Administration — `/api/admin` (9 routes)

Huit routes protégées par `require_role("admin")`, plus une route publique — accepter une invitation, puisque l'invité n'a par définition pas encore de session.

MÉTHODE	ROUTE	RÔLE
`GET`	`/users`	Liste paginée des membres de l'organisation

MÉTHODE	ROUTE	RÔLE
`POST`	`/users/invite`	Crée (ou ré-invite) un compte et envoie le lien
`PATCH`	`/users/{id}/role`	Change le rôle
`PATCH`	`/users/{id}/status`	Active / désactive
`POST`	`/users/{id}/reset-webauthn`	Révoque toutes les passkeys
`GET`	`/users/{id}/project-access`	Projets attribués
`PUT`	`/users/{id}/project-access`	Remplace l'ensemble des attributions
`GET`	`/audit-logs`	Journal d'administration, filtrable
`POST`	`/invitations/{token}/accept`	**Publique** — choisit le mot de passe et active le compte

8. Le tableau de bord

8.1 Les 9 routes de l'application

ROUTE	FICHIER	PROTÉGÉE	CONTENU
`/`	`app/page.tsx`	oui	Tableau de bord : KPI, courbe d'activité, carte de conformité
`/alerts`	`app/alerts/page.tsx`	oui	Table des alertes, filtres, triage, panneau de détail
`/projects`	`app/projects/page.tsx`	oui	Création, jeton, rotation, politique, exceptions
`/security`	`app/security/page.tsx`	oui	2FA : passkeys, TOTP, codes de récupération
`/settings`	`app/settings/page.tsx`	oui	Profil, mot de passe, journal de triage
`/admin`	`app/admin/page.tsx`	oui (`admin`)	Comptes, rôles, accès projets, journal d'administration
`/login`	`app/login/page.tsx`	non	Connexion en plusieurs étapes
`/signup`	`app/signup/page.tsx`	non	Auto-inscription + création d'organisation
`/accept-invite`	`app/accept-invite/page.tsx`	non	Activation d'un compte invité
`/forgot-password`	`app/forgot-password/page.tsx`	non	Formulaire (sans effet aujourd'hui)

8.2 Les composants métier notables

COMPOSANT	LIGNES	CE QU'IL FAIT
`users-manager.tsx`	616	Écran d'administration complet : invitation, rôle, statut, accès projets
`projects-manager.tsx`	520	Cycle de vie d'un projet, affichage unique du jeton
`dashboard-live.tsx`	397	Orchestration du tableau de bord, rafraîchissement silencieux toutes les 30 s
`alerts-data-table.tsx`	389	Table TanStack : tri, pagination, sélection
`security-manager.tsx`	384	Enrôlement passkey, QR TOTP, codes de récupération
`scan-policy-manager.tsx`	294	Seuil de blocage et exceptions de règles
`alerts-manager.tsx`	289	Filtres, recherche, actions de triage

COMPOSANT	LIGNES	CE QU'IL FAIT
`admin-audit-log.tsx`	284	Journal d'administration
`finding-detail-sheet.tsx`	274	Panneau pédagogique d'un finding
`site-header.tsx`	217	En-tête, navigation, menu utilisateur
`audit-log-viewer.tsx`	187	Journal de triage
`mfa-login-step.tsx`	184	Étape de second facteur à la connexion
`account-settings.tsx`	178	Profil et mot de passe
`app-sidebar.tsx`	118	Navigation latérale
`activity-chart.tsx`	89	Courbe des 7 derniers scans (recharts)
`recovery-codes-dialog.tsx`	86	Affichage unique des 8 codes
`auth-shell.tsx` / `auth-provider.tsx` / `auth-guard.tsx`	73 / 62 / 35	Coquille d'authentification, contexte, garde de route
`compliance-card.tsx` / `kpi-cards.tsx` / `severity-badge.tsx` / `zenith-logo.tsx`	55 / 51 / 53 / 45	Présentation

8.3 Les modules `lib/`

MODULE	LIGNES	RÔLE
`auth-api.ts`	209	`request<T>()` — **le point de passage de tous les appels**. Impose `credentials: "include"` (sans quoi le cookie httpOnly ne partirait pas), normalise les erreurs en `ApiError`, gère le `204` et les listes d'erreurs Pydantic
`platform-api.ts`	227	Projets, scans, findings, politiques. Types miroirs des schémas Pydantic
`admin-api.ts`	164	Routes d'administration
`finding-explainer.ts`	190	Base de connaissances : par `rule_id` connu (`CKV_AWS_20` , `CKV_AWS_21` , `CKV_AWS_24` , `CKV_AWS_25` …), sinon repli par catégorie. Fournit *impact*, *scénario d'attaque*, *correctif*
`i18n.tsx`	169	FR / arabe pour les écrans d'authentification, RTL posé par `AuthShell`
`data.ts`	253	Types partagés et données de démonstration
`utils.ts`	6	`cn()` — fusion de classes Tailwind

8.4 Le panneau de détail d'un finding

C'est l'endroit où le produit cesse d'être un tableau et devient un outil pédagogique. Pour un finding donné, il présente :

1. **Ce que c'est** — le problème en une phrase.
2. **Ce qu'un attaquant peut en faire** — un scénario concret. Exemple pour `CKV_AWS_24` (SSH ouvert à `0.0.0.0/0`) : *« C'est la première chose que scannent les attaquants automatisés : ils tentent des connexions par force brute 24 h/24 dès qu'un port 22 public est détecté. »*
3. **Comment corriger** — la remédiation du scanner si elle existe, sinon celle de la base de connaissances.
4. **Où** — fichier, lignes, et lien vers la ligne exacte **au commit scanné** sur GitHub.
5. **Que décider** — les quatre actions de triage, avec champ de justification.

9. Les algorithmes et règles métier

9.1 L'empreinte stable (`fingerprint`)

```
parts = [source, rule_id, file or "", str(line_start or ""), resource or ""]
fingerprint = sha1("|".join(parts).encode("utf-8")).hexdigest()
```

Ce qui entre dans l'empreinte : l'*identité* du problème. **Ce qui n'y entre pas** : `title`, `severity`, `remediation`, `raw` — tout ce qui peut changer quand le scanner est mis à jour sans que le problème change.

C'est ce qui permet à un problème redétecté au scan suivant de rester **le même problème** — et donc de conserver la décision de triage prise dessus.

Limite connue et assumée : `line_start` entre dans le calcul. Ajouter une ligne au-dessus d'un problème décale sa ligne et crée donc une **nouvelle** empreinte — la décision de triage est perdue. Le compromis est délibéré : sans le numéro de ligne, deux occurrences distinctes de la même règle dans un même fichier se confondraient.

9.2 Le score de conformité

Calculé **à la lecture, jamais stocké** : on peut ajuster les poids sans re-scanner tout l'historique.

```
score = max(0, 100 - somme(nombre_de_findings[gravite] x poids[gravite]))
```

GRAVITÉ	POINTS RETIRÉS
`critical`	25
`high`	10
`medium`	3
`low`	1
`info`	0

GRADE	SEUIL
A	≥ 90
B	≥ 75
C	≥ 60
D	≥ 40
F	< 40

Les findings couverts par une exception de règle (`ignored_rules`) sont **exclus du calcul** mais restent visibles dans la liste.

Limite reconnue : à 25 points par finding critique, quatre secrets exposés suffisent à atteindre 0. Le score sature et cesse alors de discriminer. La révision est listée dans les travaux en cours du README.

9.3 La vue « dernier état »

`GET /api/alerts` ne renvoie **pas** tout l'historique. Pour chaque projet accessible, elle prend le **dernier scan** (`ORDER BY scanned_at DESC LIMIT 1`) et ne montre que ses findings. Sans cela, un problème redétecté à chaque scan apparaîtrait en double, en triple, en dix exemplaires.

Le tri final : **plus grave d'abord**, puis le plus récemment détecté.

```
rows.sort(key=lambda r: (SEVERITY_WEIGHTS.get(r.severity, 0), r.id), reverse=True)
```

Le poids du score sert ici d'ordre de gravité — une réutilisation économique, mais qui couple le tri au barème.

9.4 La politique de blocage

```
def exceeds_threshold(by_severity, threshold):
    if not threshold or threshold not in SEVERITY_ORDER:
        return False                        # politique desactivee
    idx = SEVERITY_ORDER.index(threshold)  # ["info","low","medium","high","critical"]
    return any(by_severity.get(sev, 0) > 0 for sev in SEVERITY_ORDER[idx:])
```

Lu en français : *« bloquer dès qu'un finding atteint cette gravité ou pire »*. `fail_on_severity = "high"` bloque donc sur `high` **et** sur `critical` .

`fail_on_severity = null` désactive la politique — c'est la valeur par défaut, rétro-compatible : un projet existant ne se met pas à bloquer tout seul.

Les exceptions de règles sont retirées **avant** l'évaluation du seuil : une règle acceptée comme risque connu ne bloque plus le pipeline.

9.5 Le RBAC à cinq rôles

RÔLE	ACCÈS GLOBAL AUX PROJETS DE L'ORGANISATION	ACCÈS AUX ROUTES `/API/ADMIN/*`
`admin`	oui	oui
`security_lead`	oui	non
`developer`	non — attribution explicite	non
`auditor`	non — attribution explicite	non

RÔLE	ACCÈS GLOBAL AUX PROJETS DE L'ORGANISATION	ACCÈS AUX ROUTES `/API/ADMIN/*`
`viewer`	non — attribution explicite	non

Deux mécanismes **distincts**, qu'il ne faut pas confondre :

1. **Le contrôle de rôle** — `require_role("admin")`. Le rôle est **relu en base à chaque requête**, jamais lu dans le JWT. Un rôle retiré prend effet immédiatement, sans liste de révocation.
2. **Le cloisonnement des projets** — `list_accessible_projects` / `get_accessible_project` :

```
if user.has_global_project_access:
    # tous les projets possedes par un membre de la meme organisation
    return select(Project).where(Project.owner_id.in_(
        select(User.id).where(User.org_id == user.org_id)))
# sinon : ses propres projets + ceux explicitement partages
shared = select(ProjectAccess.project_id).where(ProjectAccess.user_id == user.id)
return select(Project).where(
    (Project.owner_id == user.id) | (Project.id.in_(shared)))
```

Le détail qui compte : hors périmètre, la réponse est **404, jamais 403**. Un 403 confirmerait l'existence de la ressource. De l'extérieur, « ça n'existe pas » et « tu n'y as pas accès » doivent être indiscernables.

9.6 Les règles de protection des comptes

Codées dans `admin.py`, et testées :

RÈGLE	CODE RENVOYÉ	POURQUOI
Le dernier admin actif ne peut pas être rétrogradé	`409`	Sinon l'organisation devient ingérable
Le dernier admin actif ne peut pas être désactivé	`409`	Idem
On ne peut pas désactiver son propre compte	`409`	Évite l'auto-exclusion accidentelle
Un admin ne gère que les membres de son organisation	`404`	Cloisonnement inter-clients
Maximum 10 invitations par heure et par admin	`429`	Anti-spam, compté sur les entrées `user.invited` du journal

`_active_admin_count` compte les admins **réellement actifs** : `role == "admin"` **et** `is_active` **et** `hashed_password IS NOT NULL`. Un admin invité mais qui n'a jamais activé son compte ne compte pas — sans ce détail, inviter un admin puis désactiver le seul admin réel verrouillerait l'organisation.

9.7 Le triage

Quatre statuts : `open`, `acknowledged`, `resolved`, `false_positive`.

- `open` est l'**état implicite** : aucune ligne n'est créée tant qu'aucune décision n'est prise. Repasser à `open` **supprime** la ligne de décision.
- Chaque changement effectif écrit une entrée dans `triage_audit_log`. Une décision identique à la précédente n'écrit rien (`if previous_status == payload.status: return`).
- La décision est indexée par empreinte : elle survit au scan suivant.

9.8 Anti-force brute et anti-énumération

MESURE	IMPLÉMENTATION
Verrouillage	5 échecs → verrou de 15 minutes, puis le compteur repart à zéro
Message unique	Même `401` et même texte pour « compte inconnu », « mauvais mot de passe » et « invitation non acceptée »
Défense temporelle	Sur un e-mail inconnu, le serveur **hache quand même** un mot de passe factice. Sans cela, une réponse instantanée trahirait l'absence de compte
`/api/auth/methods`	Un e-mail inconnu répond exactement comme un compte à mot de passe seul
Compte désactivé	`403` explicite — mais **après** validation du mot de passe, donc sans fuite

9.9 Les garde-fous de démarrage en production

`Settings._verifier_secrets`, validateur Pydantic exécuté à l'import. Si `ENVIRONMENT=production` :

CONDITION	EFFET
`SECRET_KEY` absente, égale à `dev-secret-change-me`, ou < 32 caractères	**Refus de démarrer** avec un message qui donne la commande de génération
`COOKIE_SECURE` désactivé	**Refus de démarrer**

En développement, l'application démarre mais journalise un avertissement visible.

Le raisonnement, écrit dans le code : *« Le code de ce dépôt est public : la clé par défaut l'est donc aussi. Démarrer en production avec elle laisserait n'importe qui forger un jeton de session valide — sans mot de passe ni second facteur. Mieux vaut un démarrage qui échoue bruyamment qu'une authentification qui semble fonctionner tout en étant contournable. »*

10. Sécurité : menace par menace

MENACE	PARADE EN PLACE	OÙ
Vol de session par XSS	Cookie `httpOnly` — JavaScript ne peut pas le lire. Jamais de jeton en `localStorage`	`api/cookies.py`
CSRF	`SameSite=Lax` + liste CORS explicite (jamais `*`, incompatible avec `allow_credentials`)	`cookies.py`, `main.py`
Cassage de mots de passe après fuite de base	Argon2id, *memory-hard*, paramètres OWASP, sel embarqué	`core/security.py`
Force brute en ligne	5 tentatives puis verrou de 15 minutes	`api/auth.py`
Énumération de comptes	Réponses et durées identiques dans tous les cas	`api/auth.py`
Contournement de la 2FA	Portée `mfa` vérifiée à chaque décodage de JWT	`core/security.py`, `api/deps.py`
Rejeu d'une assertion WebAuthn	Challenge en base, à usage unique, TTL 300 s	`services/mfa.py`
Clé WebAuthn clonée	Contrôle de progression de `sign_count`	`services/mfa.py`
Fuite d'un jeton de projet	Seul le SHA-256 est stocké ; rotation en un clic ; portée limitée à une seule route	`core/security.py`, `api/projects.py`
Écriture croisée entre clients	Le projet vient du jeton, jamais du contenu du message	`api/scans.py`
Lecture croisée entre clients	Cloisonnement par organisation / propriété / octroi, `404` hors périmètre	`api/deps.py`
Élévation de privilège par jeton périmé	Le rôle est relu en base à chaque requête	`api/deps.py`
Verrouillage total de l'organisation	Le dernier admin actif est protégé	`api/admin.py`
Déploiement avec la clé de développement	Refus de démarrer en production	`core/config.py`
Déni de service au login	Mot de passe borné à 1 024 octets	`core/security.py`
Troncature bcrypt	Refus au-delà de 72 octets plutôt que troncature silencieuse	`core/security.py`
Fuite d'un secret dans le rapport de scan	`gitleaks --redact` : la valeur est masquée avant l'envoi	workflow
Perte de traçabilité par suppression	Journaux append-only et dénormalisés	`models/audit_log.py`, `models/admin_audit_log.py`
Perte d'accès après perte d'appareil	8 codes de récupération à usage unique + réinitialisation admin des passkeys	`services/mfa.py`, `api/admin.py`

10.1 Ce qu'il faut annoncer comme limites

Une présentation honnête cite ces points :

- **Pas de jeton de rafraîchissement ni de révocation de session.** Un JWT de session vole 24 h. Le rôle étant relu en base, une élévation de privilège est impossible, mais une session volée reste utilisable jusqu'à expiration.
- **Pas de limitation de débit globale.** Seuls le login (verrouillage par compte) et l'invitation (10/h) sont protégés. Aucune protection sur `POST /api/scans`.
- `raw` **conserve l'objet brut du scanner.** Gitleaks masque les secrets (`--redact`), mais un autre outil mal configuré pourrait y déposer une donnée sensible.
- **Pas de chiffrement au repos.** Le secret TOTP est stocké en clair en base — contrairement aux clés WebAuthn, dont seule la partie publique existe côté serveur.
- `create_all` **au démarrage.** Aucun contrôle de version de schéma ; une évolution de modèle sur une base existante ne sera pas appliquée.

11. La méthode de travail

11.1 Le principe directeur : le code explique ses décisions

C'est la caractéristique la plus visible du dépôt et elle est systématique. Chaque module s'ouvre sur un docstring qui répond à **« pourquoi ce fichier existe et quel piège il évite »**, pas à « ce que fait ce fichier » — que le code dit déjà.

Trois exemples, tirés tels quels :

> *« SHA-256 (sans sel) suffit ici : le token est long et aléatoire (256 bits), donc non attaquable par dictionnaire. »* — `core/security.py`
>
> *« On répond 404 (et non 403) quand la ressource est hors de portée : un 403 confirmerait son existence. »* — `api/deps.py`
>
> *« PAS de `continue-on-error` ni de try/except silencieux ici : si l'envoi échoue, le job DOIT échouer — un pipeline vert alors que le scan s'est perdu en route est pire que pas de scan du tout. »* — `scan-standalone.yml`

La règle appliquée : un commentaire est écrit quand une décision non évidente a été prise, ou quand un choix protège d'un piège précis. Jamais pour paraphraser la ligne suivante.

11.2 Conventions de code

Backend (Python)

- `from __future__ import annotations` en tête de chaque module.
- Annotations de type partout, y compris les retours.
- SQLAlchemy en style 2.0 : `Mapped[...]`, `mapped_column`, `select()` — pas de `Query` hérité.
- Séparateurs visuels normalisés entre grandes sections d'un fichier : `# ------- #` sur 79 colonnes.
- Noms de fonctions en anglais, docstrings et commentaires en français.
- Une exception métier dédiée par domaine plutôt qu'un `ValueError` générique : `MfaError`, `PasswordTooLongError`.
- Les constantes de configuration sont nommées, jamais en dur : `MAX_PASSWORD_BYTES`, `CHALLENGE_TTL_SECONDS`, `RECOVERY_CODE_COUNT`, `MIN_SECRET_KEY_LENGTH`.

Frontend (TypeScript)

- Alias d'import `@/` vers `src/`.
- Les types de `lib/*-api.ts` **miroitent** les schémas Pydantic du backend. Le commentaire le dit explicitement pour que la synchronisation soit un geste conscient.
- Un seul point de passage réseau : `request<T>()` de `auth-api.ts`. Aucun `fetch` direct ailleurs — ce qui garantit que `credentials: "include"` et la normalisation des erreurs ne peuvent pas être oubliés.
- Composants métier à la racine de `components/`, primitives shadcn dans `components/ui/`.

Git

- Messages en français, à l'impératif, décrivant l'intention : *« Refuser le démarrage en production avec une configuration dangereuse »* — pas « fix config ».
- Commits de gros grain, correspondant à une capacité livrée.

11.3 Les arbitrages d'ingénierie, et leur logique

Le dépôt applique une doctrine repérable. Cinq principes :

1. Échouer bruyamment plutôt que fonctionner à moitié. Refus de démarrer en production avec une clé faible ; échec du job si l'envoi du rapport échoue ; refus d'un mot de passe de plus de 72 octets sur bcrypt plutôt que troncature silencieuse.

2. Choisir la primitive selon la menace, pas par habitude. Argon2id pour ce qu'un humain choisit, SHA-256 pour ce qu'une machine génère, SHA-1 pour un identifiant qui n'est pas un secret. Chaque choix est justifié en commentaire (§4.8).

3. Dériver plutôt que stocker. `User.status`, `has_mfa`, `webauthn_enabled` sont des propriétés calculées. Le score de conformité est recalculé à la lecture. Rien à resynchroniser, rien à re-scanner quand un barème change.

4. Dénormaliser là où la traçabilité l'exige. Les journaux d'audit copient `project_name`, `owner_id`, `finding_title`. C'est une entorse délibérée à la normalisation : un journal ne doit pas dépendre de la survie de ce qu'il documente.

5. Rendre les défauts rétro-compatibles. `fail_on_severity = null` par défaut : un projet existant ne se met pas à bloquer tout seul. Les hash bcrypt continuent d'être acceptés et migrent d'eux-mêmes.

11.4 La stratégie de test

- **Tests d'API bout en bout** via `TestClient`, sur une base SQLite **en mémoire** recréée à chaque test (`conftest.py`). Aucun test ne dépend de l'état laissé par un autre.
- **Tests unitaires purs** pour la logique de calcul : `compute_score` et `exceeds_threshold` se testent sans base ni HTTP.
- **Un test par règle métier**, nommé en langage naturel : `test_cannot_demote_last_active_admin`, `test_user_without_project_access_sees_nothing`, `test_mfa_token_cannot_open_session`.
- Les cas négatifs sont testés autant que les positifs : révocation d'accès, dépassement de quota, portée de jeton incorrecte.
- **Commande** : `cd backend && pytest`. Résultat au moment de la rédaction : **151 cas passants, 0 échec**.

11.5 La méthode de documentation

Quatre documents, quatre publics, aucune redondance (§ *Carte des documents*). Le principe suivi :

- **Toute affirmation chiffrée est vérifiée dans le code** avant d'être écrite. Le nombre de routes de ce document, par exemple, a été recompté sur les décorateurs `@router.*`.
- **Les écarts entre documentation et code sont signalés**, pas masqués. `SYNTHESE_ANALYSE.md` leur consacre une section entière.

- **Ce qui n'est pas fait est écrit** aussi clairement que ce qui l'est (§15). Un lecteur ne doit pas découvrir une lacune en lisant le code.

11.6 Le cycle de développement observable

D'après l'historique Git, le projet a été construit par capacités successives :

DATE	COMMIT	CAPACITÉ AJOUTÉE
14/07/2026	`d12824a`	Démo Terraform vulnérable + pipeline iac-guard
12/08/2026	`1690db0`	Correction d'interface (menu déroulant des projets)
22/08/2026	`ad45932`	Gestion des comptes par rôle, analyse SAST, documentation
22/08/2026	`a2854f1`	Garde-fous de démarrage en production

La progression va de la démonstration vers le durcissement — la dernière chose ajoutée est un mécanisme qui **empêche** un déploiement dangereux.

12. Procédures opérationnelles

12.1 Installer l'environnement de développement

Prérequis : Docker et Node.js 20.

```
# 1. API + base PostgreSQL
docker compose up -d --build
docker compose logs -f api        # suivre le demarrage
curl http://localhost:8000/health # doit repondre {"status":"ok",...}

# 2. Tableau de bord
cd frontend
npm install
npm run dev                       # http://localhost:3000
```

Points d'entrée : API sur `http://localhost:8000` (documentation interactive sur `/docs`), tableau de bord sur `http://localhost:3000`.

12.2 Développer sans Docker

```
cd backend
pip install -r requirements-dev.txt
uvicorn app.main:app --reload
pytest
```

La base par défaut est alors SQLite (`./iacguard.db`), aucun service à lancer. Pour PostgreSQL, renseigner `DATABASE_URL` dans un `.env` (modèle commenté : `backend/.env.example`).

12.3 Connecter un dépôt client

1. **Créer le projet** depuis le tableau de bord (`/projects`) et **copier le jeton affiché**. Il ne sera plus jamais montré.
2. **Copier deux choses** dans le dépôt à analyser : - `pipeline/github-actions-template/scan-standalone.yml` → `.github/workflows/` ; - le contenu de `pipeline/aggregator/` → un dossier `.iacguard/`.
3. **Déclarer les paramètres** dans GitHub, sous *Settings* → *Secrets and variables* → *Actions* : - secret `IAC_GUARD_TOKEN` = le jeton du projet ; - variable `IAC_GUARD_API_URL` = l'URL publique de l'API, **sans slash final**.
4. **Activer le blocage** : protéger la branche principale en exigeant le *check* `scan`. Sans cette étape, le pipeline signale mais n'empêche rien.

5. **Vérifier** : déclencher un run avec le bouton « Run workflow », puis contrôler que le scan apparaît dans `/alerts`.

12.4 Faire tourner un jeton de projet

À faire dès qu'un jeton a pu fuiter (dépôt public, journal de CI, capture d'écran).

1. Tableau de bord → `/projects` → le projet → **Régénérer le jeton**.
2. Copier le nouveau jeton (affiché une seule fois).
3. Le remplacer dans *Settings* → *Secrets and variables* → *Actions* du dépôt client.
4. Déclencher un run pour vérifier.

L'ancien jeton est invalidé immédiatement : `token_hash` est écrasé, et l'authentification échoue en `401` (« Token projet inconnu ou révoqué »).

12.5 Inviter un collaborateur

1. `/admin` → **Inviter** → e-mail + rôle.
2. Le compte est créé aussitôt, en statut `pending_invitation` (`hashed_password = NULL`).
3. Le lien d'activation est **journalisé par l'API** (voir §4.10 — l'envoi d'e-mail n'est pas branché). Le récupérer avec : `bash docker compose logs api | grep -i invitation`
4. L'invité ouvre le lien, choisit un mot de passe : le compte passe `active`.
5. Le lien expire après 72 heures. Ré-inviter réémet un jeton.

Quota : 10 invitations par heure et par admin, comptées sur les entrées `user.invited` du journal d'administration.

12.6 Donner accès à un projet

Pour un rôle **sans** accès global (`developer`, `auditor`, `viewer`) :

1. `/admin` → l'utilisateur → **Accès aux projets**.
2. Cocher les projets, valider. C'est un `PUT` : la liste envoyée **remplace** l'ancienne.
3. L'action est tracée dans le journal d'administration.

Rappel : sans aucune ligne d'accès, un rôle non global ne voit **aucun** projet. Un `admin` ou un `security_lead` voit tous les projets de l'organisation sans configuration.

12.7 Traiter une alerte

1. `/alerts` → filtrer par gravité, projet, catégorie ou statut.
2. Ouvrir le panneau de détail : impact, scénario d'attaque, correctif, lien vers la ligne au commit scanné.

3. Décider : - **Prise en compte** — vue, correction planifiée ; - **Résolue** — corrigée ; - **Faux positif** — avec justification.
4. La décision est enregistrée par empreinte : elle survit au scan suivant.
5. Elle apparaît dans le journal de triage (`/settings`).

Si une règle entière n'a pas lieu d'être sur ce projet (bucket public volontaire pour un CDN, par exemple), préférer une **exception de règle** dans `/projects` → *Politique* : les findings restent visibles mais ne comptent plus dans le score ni dans le blocage — et la justification est conservée pour l'audit.

12.8 Déployer en production

Liste de contrôle avant démarrage — l'application refuse de démarrer si les deux premiers points ne sont pas remplis :

```
# 1. Generer une cle de signature
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

#	VARIABLE	VALEUR EXIGÉE
1	`SECRET_KEY`	≥ 32 caractères, jamais `dev-secret-change-me` . **Refus de démarrer sinon.**
2	`COOKIE_SECURE`	`true` . **Refus de démarrer sinon.**
3	`ENVIRONMENT`	`production`
4	`DATABASE_URL`	`postgresql+psycopg://…`
5	`CORS_ORIGINS`	Le domaine réel du tableau de bord, liste explicite
6	`WEBAUTHN_RP_ID`	Le domaine du **frontend** (pas celui de l'API)
7	`WEBAUTHN_ORIGINS`	L'origine exacte, en `https://`
8	`FRONTEND_URL`	Base des liens d'invitation
9	`NEXT_PUBLIC_API_URL`	Côté frontend : URL publique de l'API

Points restant à traiter avant un vrai déploiement (§15) : les migrations Alembic à la place de `create_all` , un service d'e-mail transactionnel, une limitation de débit sur `POST /api/scans` .

12.9 Sauvegarder et restaurer

```
# Sauvegarde PostgreSQL
docker compose exec db pg_dump -U iacguard iacguard > sauvegarde-$(date +%F).sql

# Restauration
cat sauvegarde-2026-09-15.sql | docker compose exec -T db psql -U iacguard -d iacguard
```

`docker compose down` conserve le volume `pgdata`. `docker compose down -v` **l'efface**.

12.10 Diagnostiquer une panne

SYMPTÔME	CAUSE PROBABLE	VÉRIFICATION
Le job GitHub s'arrête sur « API injoignable »	Tunnel arrêté, URL périmée, `\r\n` collé dans la variable	`curl {API_URL}/health`
`401 Token projet inconnu ou révoqué`	Jeton tourné et non mis à jour côté GitHub	Comparer le préfixe affiché dans `/projects`
Le scan arrive mais le pipeline reste vert	`fail_on_severity` est `null`, ou le *check* `scan` n'est pas exigé par la protection de branche	`/projects` → *Politique* ; réglages GitHub
Le tableau de bord affiche « Impossible de joindre le serveur »	API arrêtée, ou origine absente de `CORS_ORIGINS`	Console du navigateur, `docker compose logs api`
L'API refuse de démarrer	Garde-fou de production déclenché	Lire le message : il nomme la variable fautive
L'enrôlement WebAuthn échoue	`WEBAUTHN_RP_ID` ne correspond pas au domaine du frontend	Comparer les deux valeurs
Un code TOTP valide est refusé	Dérive d'horloge > 30 s	Synchroniser l'horloge du téléphone

13. Tests

13.1 Résultat

```
$ cd backend && pytest
.................................................................... [ 47%]
.................................................................... [ 95%]
.......                                                              [100%]
151 passed
```

131 fonctions de test, 151 cas exécutés (l'écart vient des tests paramétrés).

13.2 Infrastructure

`backend/tests/conftest.py` :

- base **SQLite en mémoire**, recréée pour chaque test ;
- `Base.metadata.create_all` avant, `drop_all` après ;
- `get_db` surchargé par `app.dependency_overrides` ;
- un `TestClient` FastAPI par test.

Conséquence : aucune fuite d'état entre tests, aucun besoin de base externe, exécution en quelques secondes.

13.3 Répartition

FICHIER	FONCTIONS	DOMAINE COUVERT
`test_auth.py`	30	Inscription, connexion, verrouillage, énumération, portées de JWT
`test_api.py`	25	Ingestion, lecture, cloisonnement, projets
`test_admin.py`	18	Invitations, rôles, statuts, accès projets, quotas, dernier admin
`test_2fa.py`	16	WebAuthn, TOTP, codes de récupération
`test_scan_policy.py`	9	Seuil de blocage, exceptions de règles
`test_triage.py`	7	Les quatre statuts, persistance par empreinte
`test_scoring.py`	7	Barème, grades, saturation à 0
`test_profile.py`	6	Profil, changement de mot de passe
`test_audit_log.py`	6	Journaux append-only

FICHIER	FONCTIONS	DOMAINE COUVERT
`test_security.py`	4	Hachage, migration bcrypt → Argon2, portées
`test_aggregates.py`	3	Comptages par gravité, exclusions

13.4 Ce qui n'est pas couvert

À dire explicitement plutôt que laisser découvrir :

- **Aucun test frontend** — ni unitaire, ni de composant, ni de bout en bout.
- **Aucun test de l'agrégateur** — les *parsers* des quatre formats ne sont pas testés, alors que `pipeline/aggregator/samples/` contient précisément les échantillons qui le permettraient.
- **Aucune intégration continue sur ce dépôt** — les tests existent mais rien ne les exécute automatiquement. Le projet analyse la CI des autres sans en avoir une.
- **Aucun test sur PostgreSQL** — tout tourne sur SQLite ; les écarts de comportement (fuseaux horaires, types JSON) ne sont pas exercés.

14. Configuration de référence

14.1 Backend — classe `Settings`

VARIABLE	DÉFAUT	RÔLE
`APP_NAME`	`iac-guard`	Nom affiché, titre OpenAPI
`ENVIRONMENT`	`development`	`production` active les garde-fous bloquants
`DATABASE_URL`	`sqlite:///./iacguard.db`	Base de données
`SECRET_KEY`	`dev-secret-change-me`	Signature des JWT. ≥ 32 caractères en production
`ACCESS_TOKEN_EXPIRE_MINUTES`	`1440`	Durée d'une session (24 h)
`CORS_ORIGINS`	`["http://localhost:5173","http://localhost:3000"]`	Liste explicite, jamais `*`
`SESSION_COOKIE_NAME`	`iacguard_session`	Cookie de session
`MFA_COOKIE_NAME`	`iacguard_mfa`	Cookie intermédiaire 2FA
`COOKIE_SECURE`	`false`	`true` obligatoire en production
`COOKIE_SAMESITE`	`lax`	Protection CSRF
`MFA_TOKEN_EXPIRE_MINUTES`	`5`	Durée du jeton `mfa`
`MAX_LOGIN_ATTEMPTS`	`5`	Seuil de verrouillage
`LOCKOUT_MINUTES`	`15`	Durée du verrou
`WEBAUTHN_RP_ID`	`localhost`	Domaine du **frontend**
`WEBAUTHN_RP_NAME`	`iac-guard`	Nom affiché à l'enrôlement, émetteur TOTP
`WEBAUTHN_ORIGINS`	`["http://localhost:3000"]`	Origines autorisées
`FRONTEND_URL`	`http://localhost:3000`	Base des liens d'invitation
`INVITATION_EXPIRE_HOURS`	`72`	Durée de validité d'une invitation
`MAX_INVITES_PER_HOUR`	`10`	Quota par admin

Chargement : variables d'environnement, puis fichier `.env`, `extra="ignore"`.

14.2 Frontend

VARIABLE	DÉFAUT	RÔLE
`NEXT_PUBLIC_API_URL`	`http://localhost:8000`	Base de l'API. Le slash final est retiré automatiquement

Fichier local : `frontend/.env.local`.

14.3 Ports et réseau

SERVICE	PORT	NOTES
API	8000	`/health`, `/docs`
Tableau de bord	3000	Doit figurer dans `CORS_ORIGINS`
PostgreSQL	5432	Exposé par Compose pour le développement

Dans Compose, l'API joint la base par le nom de service `db`, résolu par le DNS interne. Le démarrage de l'API attend le *healthcheck* `pg_isready`.

15. Limites connues et suite

15.1 Travaux en cours annoncés par le README

CHANTIER	ÉTAT RÉEL DANS LE CODE
Moteur de conformité CNDP (loi 09-08) / Bank Al-Maghrib	**Non commencé.** Les référentiels affichés dans l'interface sont des valeurs de démonstration. Le schéma unifié réserve la place : l'enrichissement se fera côté backend à partir de `rule_id`
Export PDF des rapports	Non implémenté
Révision du score	Le barème sature à 0 dès quatre findings critiques (§9.2)
Envoi des e-mails d'invitation	`services/email.py` journalise le lien, ne l'envoie pas
Migrations Alembic	`create_all` au démarrage, aucun contrôle de version de schéma

15.2 Autres éléments incomplets identifiés dans le code

- `POST /api/auth/forgot-password` répond toujours `202` et ne fait rien. Le formulaire existe côté interface.
- **Conftest / OPA** : parser complet, énumération présente, workflow prêt — mais `policies/` est vide et le workflow recommandé ne l'appelle pas. Le support est donc inactif en pratique.
- **Version de Checkov non épinglée** (`pip install checkov`) alors que Gitleaks et Semgrep le sont. Un scan peut changer de résultat sans qu'aucun code n'ait bougé.
- **i18n partielle** : français et arabe couvrent les écrans d'authentification ; le reste de l'application est en français seulement.

15.3 Limites techniques structurelles

- `GET /api/alerts` **fait une requête par projet** puis trie et pagine **en mémoire Python**. Correct pour quelques dizaines de projets, à revoir au-delà.
- **Le fingerprint dépend du numéro de ligne** : un décalage de ligne casse le suivi d'un problème (§9.1).
- **Pas de révocation de session** : un JWT volé reste valide jusqu'à 24 h.
- **Aucune limitation de débit sur l'ingestion** : un jeton fuité permet d'inonder la base de scans.
- **Le secret TOTP est stocké en clair.**

15.4 Ordre de priorité suggéré

1. Épingler la version de Checkov — une ligne, supprime une source de non-reproductibilité.
2. Ajouter une intégration continue qui exécute `pytest` sur ce dépôt — la crédibilité d'un outil de CI en dépend.

3. Tester l'agrégateur avec les échantillons déjà présents dans `samples/`.
4. Passer à Alembic avant tout déploiement réel.
5. Brancher un service d'e-mail transactionnel, ce qui débloque aussi la réinitialisation de mot de passe.
6. Revoir le barème du score (échelle logarithmique ou plafond par gravité).

16. Glossaire

TERME	DÉFINITION
Agrégateur	Le script Python du pipeline qui convertit les sorties des scanners en un schéma unique et les envoie à l'API.
Argon2id	Fonction de hachage de mots de passe *memory-hard*, recommandée par l'OWASP. Rend le cassage massif sur GPU coûteux.
CTAP2	Protocole entre le navigateur et l'authentificateur (clé USB, TPM), sous-jacent à WebAuthn.
Empreinte / fingerprint	Identifiant déterministe d'un problème, calculé sur son identité (outil, règle, fichier, ligne, ressource). Permet de le suivre d'un scan à l'autre.
Exception de règle	Décision d'ignorer **toute** occurrence d'une règle sur un projet, avec justification. Différent du triage.
Finding	Un problème détecté par un scanner, normalisé au schéma unifié.
HCL2	Langage de configuration de Terraform.
IaC	*Infrastructure as Code* — décrire l'infrastructure en fichiers versionnés plutôt qu'en clics dans une console.
JWT	*JSON Web Token* — jeton signé portant l'identité et la portée. Ici HS256.
Passkey	Identifiant WebAuthn découvrable, permettant de se connecter sans saisir d'e-mail.
Portée / scope	Champ du JWT distinguant une session complète (`session`) d'une étape 2FA (`mfa`).
RBAC	*Role-Based Access Control* — droits attribués par rôle. Ici cinq rôles.
Rego	Langage de règles d'Open Policy Agent, utilisé par Conftest.
SAST	*Static Application Security Testing* — analyse du code source sans l'exécuter.
Scan	Un rapport reçu du pipeline pour un projet, à un commit donné.
Seuil de blocage	`fail_on_severity` : gravité à partir de laquelle le pipeline échoue.
Sign count	Compteur incrémenté par un authentificateur WebAuthn à chaque usage. Sa régression signale un clonage.
TOTP	*Time-based One-Time Password* (RFC 6238) — le code à 6 chiffres qui change toutes les 30 secondes.
Triage	Décision d'un analyste sur une alerte : ouverte, prise en compte, résolue, faux positif.
WebAuthn	Standard W3C d'authentification par clé cryptographique liée à l'appareil, résistante au *phishing*.

Document rédigé le 15 septembre 2026 à partir d'une relecture du dépôt à l'état du commit `a2854f1`. Chaque chiffre cité a été vérifié dans le code ; la suite de tests a été exécutée pour valider le résultat annoncé.